Problem Set 6

Due March 6, 2026 at 11:59 PM Pacific

Turn in your problem set by Gradescope. Include any code you write in your solutions as well. Mark down how much time you spent on the homework set.

1. **Point Cloud Alignment:** Consider two point clouds $(r_1, \ldots, r_N)$ and $(\hat{r}_1, \ldots, \hat{r}_N)$, where $r_i, \hat{r}_i \in \mathbb{R}^3$ for all i. We would like to translate and rotate the second point cloud so that it best matches with the first. Assume both clouds have the same number of points and we know which point in the first cloud corresponds to which point in the second (these assumptions greatly simplify the problem and are the focus of current research). We would like to find a rotation R and translation τ that minimizes the following loss

$$J(R, \tau) = \frac{1}{N} \sum_{i=1}^{N} ||R\hat{r}_i + \tau - r_i||_2^2$$

subject to

$$RR^T = I$$

This is a non-convex optimization problem due to the orthogonality constraint $RR^T = I$. However, we can still solve this problem optimally. Let's seek to solve this optimization problem through two ways.

   (a) Solve for the optimal $\tau*$ as a function of $R, \hat{r}_i, r_i$. This should just be the solution to an ordinary least-squares problem. In terms of properties of the two point clouds, what does τ physically represent?

   (b) Solve for the rotation R^* by treating it as a linear optimization problem. In the process, also show that the rotation matrix you solved is globally optimal (i.e. there doesn't exist another rotation matrix that can have a lower loss).

   Hint: The Singular Value Decomposition $A = U\Lambda V^T$, where U, V are orthogonal and Λ is diagonal, may come in handy. All matrices have an SVD decomposition.

   (c) Show that your answer in part (b) is equivalent to taking the SVD of a simpler 3×3 matrix $H = \sum_j^N (\hat{r} - \hat{r}_c)(r - r_c)^T$.

   Remark: Part (c) is important because you will need a fast implementation of getting the rotation for Problem 3.

   (d) Align two point clouds, one the ground-truth `ground-truth.ply` and the other, a rotated, translated point cloud with additive noise `transformed-associated.ply`. What's the loss $J(R_t^*, \tau_t^*)$?

   (e) What's the SE(3) error between your optimized pose and the ground-truth pose (located in `gt-pose.csv`)? The CSV file stores the pose as a quaternion (x, y, z, w) and the translation (px, py, pz) in that order. Report both the rotational error (in degrees), and the translational error.

   (f) Is it possible that the program is optimally solved, but the R matrix of the solution does not correspond to a physical rotation? In which case, what does R physically do to the point cloud? (Aside: even when R is not a rotation matrix, there are methods to still extract a valid rotation).

2. **Bundle Adjustment SLAM in 2D:** In this problem you will use a standard open source SLAM package, GTSAM, to perform a bundle adjustment SLAM. (Recall, bundle adjustment SLAM solves for both landmark locations and robot poses at the same time. Alternatively, Pose Graph Optimization (PGO) SLAM pre-processes raw sensor measurements in a "front end", and optimizes only robot poses in a "back end.") We will use the Python interface to the GTSAM library, which is implemented in C++. We use a sparse 2D problem setup to ensure quick optimization in Google Colab. If you choose to consider dense or 3D SLAM for your final project, you will likely not want to use Google Colab, and you would likely use a PGO style SLAM.

(a) Draw a simple factor graph by hand or with a computer program, such as PowerPoint, that meets the following criteria.

- Three pose states x_0, x_1, and x_2.
- Two landmarks, ℓ_0 and ℓ_1, marked by a 2D position.
- Four range and bearing measurements as follows:
  - $y^{rb}_{0,0}$: between pose x_0 and landmark ℓ_0
  - $y^{rb}_{0,1}$: between pose x_0 and landmark ℓ_1
  - $y^{rb}_{1,1}$: between pose x_1 and landmark ℓ_1
  - $y^{rb}_{2,1}$: between pose x_2 and landmark ℓ_1
- Odometry measurements $y^{o}_{i,i+1}$ between successive poses.
- Loop closure constraint as odometry-like measurement $y^{lc}_{0,2}$ between pose states x_0 and x_2.

We now consider a more complex scenario. We will use the robot trajectory and landmarks in Figure 1. You will see three data files on Canvas. In the file `robot_history_5.csv`, we provide the ground truth history of the robot pose and measurements for a maximum sensor range of 5 meters. We will add noise and reconstruct the robot pose and landmark locations. The following steps will guide you in constructing the factor graph in GTSAM.

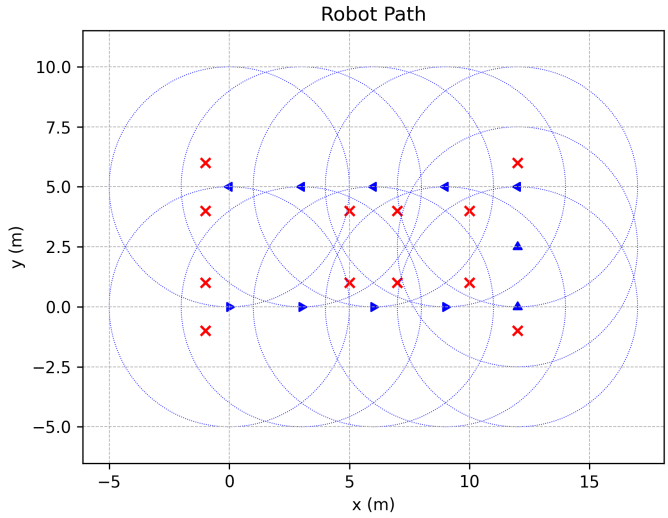


Figure 1: Robot Ground Truth Path. The landmarks are indicated as a red '×', the robot is a blue triangle oriented in the direction of travel, and the dotted blue circles indicate the maximum sensor range.

Please refer to the provided Google Colab at the link below (using your Stanford login), though you do not need to use Google Colab if you prefer to install GTSAM locally.

`https://colab.research.google.com/drive/1tJqy1jNUeGC2V235yTdWqyOElQdIclMp?usp=sharing`

(b) Verify that you can pip install GTSAM. On Google Colab, you can run `!pip install gtsam` at the top. Please read through the provided code to load and process the files. Instantiate the factor graph with:

```
graph = gtsam.NonlinearFactorGraph()
```

This is an empty factor graph. To *add* a factor to the factor graph, we will call `graph.add(...)`. To simplify the notation, GTSAM offers shorthand variables `X` and `L` for state and landmark, respectively, to organize the SLAM factors. In the provided Google Colab, we have initialized lists for `X` and `L`. The first factor we want to add is the prior factor to initialize the map at the origin. Fill in the required code based on the factor description below.

`gtsam.PriorFactorPose2(X_start, pose, noise)`

For `X_start`, use the zero-th element of the `X` list. For `pose`, use the `gtsam.Pose2(x, y, theta)` function specifying the origin and zero angle (i.e., aligned with the x-axis).

(c) Iterate through the provided data (`Pandas` is recommended). Use the factor type below to add the relative motion factors (i.e., from the odometry or relative state between successive states).

`gtsam.BetweenFactorPose2(X_prev, X_curr, rel_pose, motion_noise)`

Use the factor type below to add the bearing and range factors (i.e., from the sensor model) per each landmark in view at the given time step.

`gtsam.BearingRangeFactor2D(X_curr, L_curr, bearing, range, noise`

For the bearing measurement, use `gtsam.Rot2.fromDegrees(bearing_angle)`. Lastly, we will use the `BetweenFactorPose2` method to add a loop closure constraint between the second and eighth elements (zero-indexed) of the `X` list with position offset of 0 horizontally, 5 vertically, and 180 degrees in rotation. How many factors are in your factor graph (print `graph` to verify)?

(d) We will initialize the values from the motion alone. Instantiate the initial estimate with

`initial_estimate = gtsam.Values()`

You can input an initial estimate with `initial_estimate.insert(X_or_L_var, pose_or_point)`. Be sure to use `gtsam.Pose2(x, y, theta)` for the pose and `gtsam.Point2(x, y)` for the landmark. Plot the initial estimates for `X` and `L` using the functions below:

`gtsam_plot.plot_pose2(...)` and `gtsam_plot.plot_point2(...)`.

Include the plot in your write-up.

(e) Run the Levenberg Marquardt in GTSAM as listed in the Google Colab. GTSAM will provide estimates of `X` and `L` through the optimizer and will provide the marginal Gaussian distributions in `gtsam.Marginals`. Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma). Include the plot in your write-up.

(f) What do you notice about the orientation of the solution and the shape and orientation of the confidence ellipses? Why does this occur?

(g) Switch out `robot_history_5.csv` for `robot_history_3.csv` (maximum sensor range of 3 meters) and `robot_history_10.csv` (maximum sensor range of 10 meters). Using the same initialization for `X` and `L`, reconstruct the factor graphs. What are the factor graph sizes in each case?

Hint: You should only have to change the file input. All the other functions you wrote should still be applicable!

(h) Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma) in each case. What do you notice about the final results at different sensor ranges? Explain your result with the connectivity of the resulting factor graphs.

1. **Point Cloud Alignment:** Consider two point clouds $(r_1, \ldots, r_N)$ and $(\hat{r}_1, \ldots, \hat{r}_N)$, where $r_i, \hat{r}_i \in \mathbb{R}^3$ for all i. We would like to translate and rotate the second point cloud so that it best matches with the first. Assume both clouds have the same number of points and we know which point in the first cloud corresponds to which point in the second (these assumptions greatly simplify the problem and are the focus of current research). We would like to find a rotation R and translation τ that minimizes the following loss

$$J(R, \tau) = \frac{1}{N} \sum_{i=1}^{N} ||R\hat{r}_i + \tau - r_i||_2^2$$

subject to

$$RR^T = I$$

This is a non-convex optimization problem due to the orthogonality constraint $RR^T = I$. However, we can still solve this problem optimally. Let's seek to solve this optimization problem through two ways.

(a) Solve for the optimal $\tau*$ as a function of $R, \hat{r}_i, r_i$. This should just be the solution to an ordinary least-squares problem. In terms of properties of the two point clouds, what does τ physically represent?

(b) Solve for the rotation R^* by treating it as a linear optimization problem. In the process, also show that the rotation matrix you solved is globally optimal (i.e. there doesn't exist another rotation matrix that can have a lower loss).
Hint: The Singular Value Decomposition $A = U\Lambda V^T$, where U, V are orthogonal and Λ is diagonal, may come in handy. All matrices have an SVD decomposition.

(c) Show that your answer in part (b) is equivalent to taking the SVD of a simpler 3×3 matrix $H = \sum_j^N (\hat{r} - \hat{r}_c)(r - r_c)^T$.
Remark: Part (c) is important because you will need a fast implementation of getting the rotation for Problem 3.

(d) Align two point clouds, one the ground-truth `ground-truth.ply` and the other, a rotated, translated point cloud with additive noise `transformed-associated.ply`. What's the loss $J(R_t^*, \tau_t^*)$?

(e) What's the SE(3) error between your optimized pose and the ground-truth pose (located in `gt-pose.csv`)? The CSV file stores the pose as a quaternion (x, y, z, w) and the translation (px, py, pz) in that order. Report both the rotational error (in degrees), and the translational error.

(f) Is it possible that the program is optimally solved, but the R matrix of the solution does not correspond to a physical rotation? In which case, what does R physically do to the point cloud? (Aside: even when R is not a rotation matrix, there are methods to still extract a valid rotation).

a) $J(R,\tau) = \frac{1}{N}\sum_{i=1}^{N} \left\| \left(R\hat{r}_i + \tau - r_i\right) \right\|^2$

$e_i = R\hat{r}_i + \tau - r_i$

$J(R,\tau) = \frac{1}{N}\sum_{i=1}^{N} \|e\|^2 = \frac{1}{N}\sum_{i=1}^{N} e_i^T e_i$

$\frac{\partial J}{\partial \tau} = \frac{1}{N}\sum_{i=1}^{N} 2e_i \frac{\partial e}{\partial \tau} = \frac{1}{N}\sum_{i=1}^{N} 2e_i = 0$

$\sum_{i=1}^{N} e_i = \sum_{i=1}^{N} R\hat{r}_i + \tau - r_i = 0$

$\sum_{i=1}^{N} R\hat{r}_i + \sum_{i=1}^{N} \tau^* + \sum_{i=1}^{N} r_i = 0 \quad = R\sum_{i=1}^{N} \hat{r}_i + N\tau^* - \sum_{i=1}^{N} r_i = 0$

$N\tau^* = \sum_{i=1}^{N} r_i - R\sum_{i=1}^{N} \hat{r}_i$

$\boxed{\tau^* = r_c - R\hat{r}_c}$

τ aligns the 2 point clouds after rotation and τ^* aligns them with the smallest loss

c) $J(R,\tau^*) = \frac{1}{N}\sum_{i=1}^{N} \left\| \left(R\hat{r}_i + \tau^* - r_i\right) \right\|^2 = \frac{1}{N}\sum_{i=1}^{N} \|R\bar{r}_i - \bar{r}_i\|$

$= \frac{1}{N}\sum_{i=1}^{N} \left[\left(R\bar{r}_i\right)^T \left(R\bar{r}_i\right) + \bar{r}_i^T\bar{r}_i - 2\left(R\hat{r}_i\right)^T \bar{r}_i \right]$

$\hat{r}_i^T R^T R \hat{r}_i = \hat{r}_i^T \hat{r}_i$ (I)

$\bar{r}_i^T R^T \bar{r}_i$

$J(R,\tau^*) = \frac{1}{N}\sum_{i=1}^{N} \left[\hat{r}_i^T\hat{r}_i + \bar{r}_i^T\bar{r}_i - 2\hat{r}_i^T R^T \bar{r}_i \right]$

minimizing J same as maximizing this term (because of negative sign)

Fixing $U\Lambda V^T$ → $a^T b = tr(ab^T)$ → $\sum \bar{r}_i^T R^T \bar{r}_i = tr\left(R\Sigma \hat{r}_i \bar{r}_i^T\right)$

H matrix where $H = U\Lambda U^T$

$tr(RH) = tr\left(R U\Lambda V^T\right) = tr\left(V^T R U \Lambda\right)$

all orthogonal; Q

for orthogonal matrices, $\|q_{ii}\| \leq 1$, so max when $q_{ii} = 1$ ∴ $Q = I$

∴ $Q = V^T R U$ $\quad R = V U^T$

$\boxed{R^* = VU^T}$

globally optimal because we set all $q_{ii} = 1$ and no matrix can produce a larger trace

c) $H = \sum_{j=1}^{N} (\hat{v}_j \cdot \hat{v}_c)(r - r_c)^T = \sum_{j=1}^{N} \bar{v}_j r_j^T$ from before

SVD $= U \Lambda V^T$ like before

use H to get U and V^T, then get $R^* = VU^T$

d)

```
Optimal translation : [0.51769478 0.57728611 0.50401143]
Optimal rotation:
[[-0.30702737 -0.9514257   0.02287657]
 [-0.89566072  0.28073815 -0.34493183]
 [ 0.32175468 -0.12639316 -0.93834892]]
Loss J(R, tau): 0.007494166321482214
```

e)

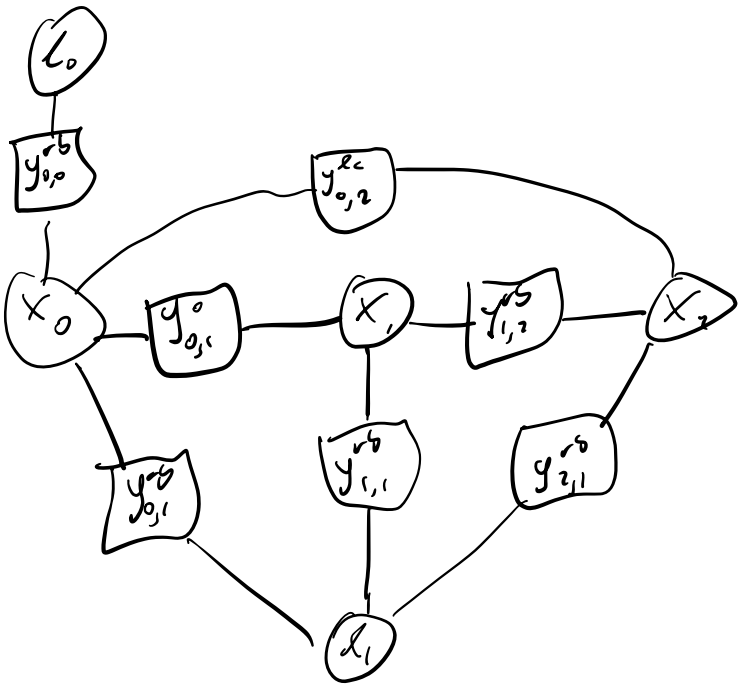
```
Rotation error: 6.168882851498211e-15º
Translation error: 1.6011864169946884e-15m
```

f) Since $R^T R = I$ we know $\det(R) = \pm 1$ but $\det(R) = -1$ flips the image, which is not physically possible. This could be handled by flipping the sign of one vector

2. **Bundle Adjustment SLAM in 2D:** In this problem you will use a standard open source SLAM package, GTSAM, to perform a bundle adjustment SLAM. (Recall, bundle adjustment SLAM solves for both landmark locations and robot poses at the same time. Alternatively, Pose Graph Optimization (PGO) SLAM pre-processes raw sensor measurements in a "front end", and optimizes only robot poses in a "back end.") We will use the Python interface to the GTSAM library, which is implemented in C++. We use a sparse 2D problem setup to ensure quick optimization in Google Colab. If you choose to consider dense or 3D SLAM for your final project, you will likely not want to use Google Colab, and you would likely use a PGO style SLAM.

(a) Draw a simple factor graph by hand or with a computer program, such as PowerPoint, that meets the following criteria.

- Three pose states x_0, x_1, and x_2.
- Two landmarks, ℓ_0 and ℓ_1, marked by a 2D position.
- Four range and bearing measurements as follows:
  - $y^{rb}_{0,0}$: between pose x_0 and landmark ℓ_0
  - $y^{rb}_{0,1}$: between pose x_0 and landmark ℓ_1
  - $y^{rb}_{1,1}$: between pose x_1 and landmark ℓ_1
  - $y^{rb}_{2,1}$: between pose x_2 and landmark ℓ_1
- Odometry measurements $y^{o}_{i,i+1}$ between successive poses.
- Loop closure constraint as odometry-like measurement $y^{lc}_{0,2}$ between pose states x_0 and x_2.



We now consider a more complex scenario. We will use the robot trajectory and landmarks in Figure 1. You will see three data files on Canvas. In the file `robot_history_5.csv`, we provide the ground truth history of the robot pose and measurements for a maximum sensor range of 5 meters. We will add noise and reconstruct the robot pose and landmark locations. The following steps will guide you in constructing the factor graph in GTSAM.

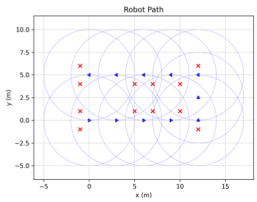


Figure 1: Robot Ground Truth Path. The landmarks are indicated as a red '×', the robot is a blue triangle oriented in the direction of travel, and the dotted blue circles indicate the maximum sensor range.

Please refer to the provided Google Colab at the link below (using your Stanford login), though you do not need to use Google Colab if you prefer to install GTSAM locally.

https://colab.research.google.com/drive/1tJqy1jNUeGC2V235yTdWqyOElQdIclMp?usp=sharing

(b) Verify that you can pip install GTSAM. On Google Colab, you can run `!pip install gtsam` at the top. Please read through the provided code to load and process the files. Instantiate the factor graph with:

```
graph = gtsam.NonlinearFactorGraph()
```

This is an empty factor graph. To *add* a factor to the factor graph, we will call `graph.add(...)`. To simplify the notation, GTSAM offers shorthand variables `X` and `L` for state and landmark, respectively, to organize the SLAM factors. In the provided Google Colab, we have initialized lists for `X` and `L`. The first factor we want to add is the prior factor to initialize the map at the origin. Fill in the required code based on the factor description below.

```
gtsam.PriorFactorPose2(X_start, pose, noise)
```

For `X_start`, use the zero-th element of the `X` list. For `pose`, use the `gtsam.Pose2(x, y, theta)` function specifying the origin and zero angle (i.e., aligned with the x-axis).

(c) Iterate through the provided data (`Pandas` is recommended). Use the factor type below to add the relative motion factors (i.e., from the odometry or relative state between successive states).

```
gtsam.BetweenFactorPose2(X_prev, X_curr, rel_pose, motion_noise)
```

Use the factor type below to add the bearing and range factors (i.e., from the sensor model) per each landmark in view at the given time step.

```
gtsam.BearingRangeFactor2D(X_curr, L_curr, bearing, range, noise
```

For the bearing measurement, use `gtsam.Rot2.fromDegrees(bearing.angle)`. Lastly, we will use the `BetweenFactorPose2` method to add a loop closure constraint between the second and eighth elements (zero-indexed) of the `X` list with position offset of 0 horizontally, 5 vertically, and 180 degrees in rotation. How many factors are in your factor graph (print `graph` to verify)?

(d) We will initialize the values from the motion alone. Instantiate the initial estimate with

```
initial_estimate = gtsam.Values()
```

You can input an initial estimate with `initial_estimate.insert(X_or_L_var, pose_or_point)`. Be sure to use `gtsam.Pose2(x, y, theta)` for the pose and `gtsam.Point2(x, y)` for the landmark. Plot the initial estimates for `X` and `L` using the functions below:

```
gtsam_plot.plot_pose2(...) and gtsam_plot.plot_point2(...).
```

Include the plot in your write-up.

(e) Run the Levenberg Marquardt in GTSAM as listed in the Google Colab. GTSAM will provide estimates of `X` and `L` through the optimizer and will provide the marginal Gaussian distributions in `gtsam.Marginals`. Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma). Include the plot in your write-up.

(f) What do you notice about the orientation of the solution and the shape and orientation of the confidence ellipses? Why does this occur?

(g) Switch out `robot_history_5.csv` for `robot_history_3.csv` (maximum sensor range of 3 meters) and `robot_history_10.csv` (maximum sensor range of 10 meters). Using the same initialization for `X` and `L`, reconstruct the factor graphs. What are the factor graph sizes in each case?
Hint: You should only have to change the file input. All the other functions you wrote should still be applicable!

(h) Plot the optimized estimates for `X` and `L` along with the 2D Gaussian Confidence Ellipse (1-sigma) in each case. What do you notice about the final results at different sensor ranges? Explain your result with the connectivity of the resulting factor graphs.

c) 57 factors

```
NonlinearFactorGraph: size: 57

Factor 0: PriorFactor on x0
  prior mean:  (0, 0, 0)
  noise model: diagonal sigmas [0.1; 0.1; 0.0174532925];

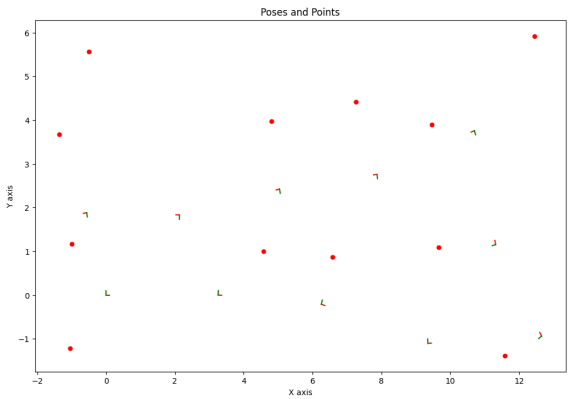
Factor 1: BetweenFactor(x0,x1)
  measured:  (3.25492435, 0, -0.0706048005)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 2: BearingRangeFactor
Factor 2:   keys = { x1 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -2.89661399
range  4.12310563

Factor 3: BearingRangeFactor
Factor 3:   keys = { x1 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.89661399
range  4.12310563

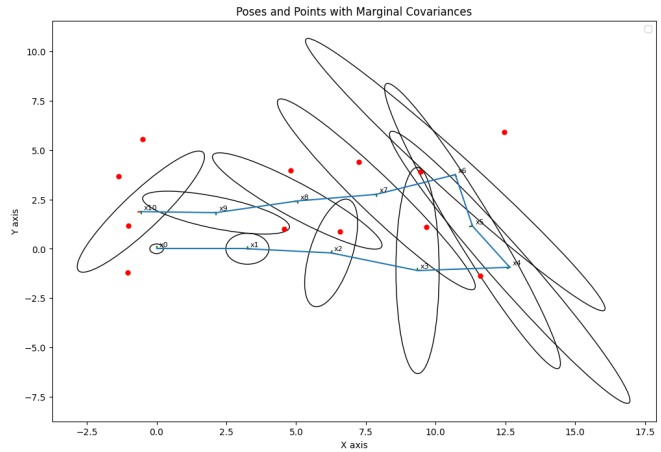
Factor 4: BearingRangeFactor
Factor 4:   keys = { x1 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
...
  measured:  (-0.233879407, 1.95151853, 2.97021382)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];
```

d)

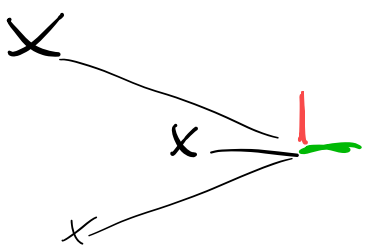


• Landmarks

e)



f) The covariance is lower when there are landmarks that are all in a line, which can provide more distance information in that direction.



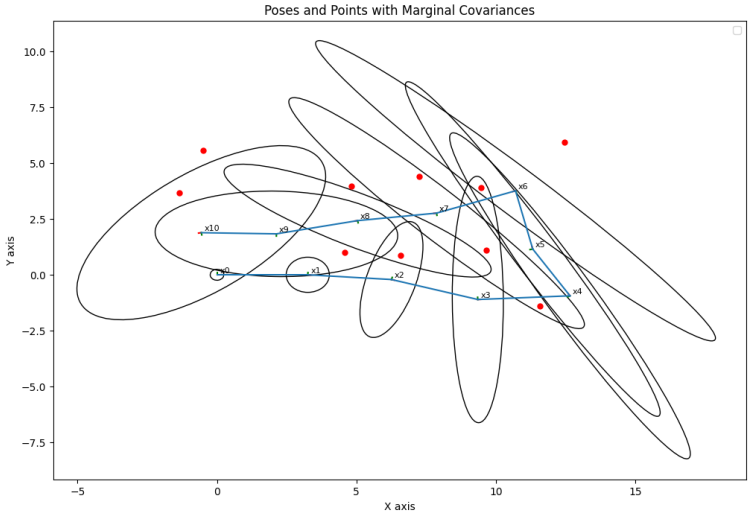
More information in given direction to decrease uncertainty

g)

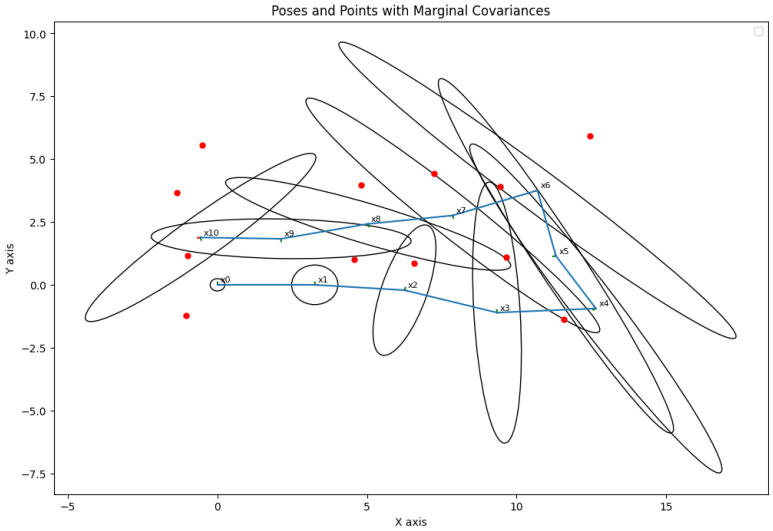
Factor graph sizes

3m → 30 nodes
5m → 57 nodes
10m → 106 nodes

3



10



h) For poses that are far from landmarks, like with 9 and 10, increasing from 3m to 10m included several additional sensors particularly in the direction along the "short" direction of the ellipses. More measurements/ factors on the factor graph allow optimizer to have more constraints and have less uncertainty

Prob 1

Part d

In [42]:
```
import open3d as o3d
import numpy as np
from scipy.spatial.transform import Rotation

pcd_gt = o3d.io.read_point_cloud("ground_truth.ply")
pcd_noisy = o3d.io.read_point_cloud("transformed_associated.ply")

r = np.asarray(pcd_gt.points)
r_hat = np.asarray(pcd_noisy.points)
```

In [43]:
```
N = len(r)

# center
r_c = np.mean(r, axis=0)
r_hat_c = np.mean(r_hat, axis=0)

# center the point clouds
r_bar = r - r_c
r_hat_bar = r_hat - r_hat_c

# Run part (b)/(c)
H = r_hat_bar.T @ r_bar
U, S, Vt = np.linalg.svd(H)
R = Vt.T @ U.T

# From part (a)
tau = r_c - R @ r_hat_c
```

In [44]:
```
# print(r_hat.shape)
```

In [45]:
```
# Align other cloud (r_hat)
# Dumb numpy stuff to get this to work vector-wise
r_transformed = (R @ r_hat.T).T

# part (a)
e = r_transformed - r + tau
loss = np.mean(np.sum(e**2, axis=1))
```

In [46]:
```
print(f"Optimal translation : {tau}")
print(f"Optimal rotation:\n{R}")
print(f"Loss J(R, tau): {loss}")
```

```
Optimal translation : [0.51769478 0.57728611 0.50401143]
Optimal rotation:
[[-0.30702737 -0.9514257   0.02287657]
 [-0.89566072  0.28073815 -0.34493183]
 [ 0.32175468 -0.12639316 -0.93834892]]
Loss J(R, tau): 0.007494166321482214
```

Part e

In [47]:
```
gt_pose = np.loadtxt("gt_pose.csv", delimiter=",", skiprows=1)

qx, qy, qz, qw, px, py, pz = gt_pose
rot_truth = Rotation.from_quat([qx, qy, qz, qw], scalar_first=False)
position = np.array([px, py, pz])
```

In [48]:
```
rot_calc = Rotation.from_matrix(R)
R_err = rot_truth * rot_calc.inv()
```

Prob 2

In [49]:
```
# First, install gtsam
# !pip3 install gtsam
```

In [50]:
```
# !pip3 list
```

In [51]:
```
# Import standard packages
# For vectorized operations and linear algebra
import numpy as np
# For plotting
import matplotlib.pyplot as plt
# For loading in the CSV data as a proper table (i.e., dataframe)
import pandas as pd

# Import the key packages for gtsam
# The top level package
import gtsam
# As implied, the plotting tools
import gtsam.utils.plot as gtsam_plot
# Convenience methods for landmarks (L) and states (X)
# These need to be reset if defined later
from gtsam.symbol_shorthand import L, X
```

Now we need to load in the data. Click the folder icon at left. This is your directory. Drag the `robot_history_{num}.csv` files into the panel that appears. You will have to repeat this if you restart the Google Colab session.

In [52]:
```
# Load in the data as a table

# You will come back to change this at the end
### TODO (Q4.b and Q4.g) ###
```

```
# filename = 'robot_history_3.csv'
# filename = 'robot_history_5.csv'
filename = 'robot_history_10.csv'
robot_gt_df = pd.read_csv(filename) # this is GROUND TRUTH (gt)!
running_dataset_3 = True if "3" in filename else False
###

# Data is (time), (x, y, theta), (speed, dtheta), (measurements)
n_time_steps, n_params = robot_gt_df.shape
n_landmarks = (n_params - (1 + 3 + 2)) // 2

print(f"Data contains {n_time_steps} time steps and {n_landmarks} landmarks.
robot_gt_df
```

```
Data contains 11 time steps and 12 landmarks.
```

Out[52]:

	# time	x	y	theta	speed	dtheta	range_0	range_1	range_2	rang
0	0.0	0.0	0.0	0.000000	3.0	0.000000	1.414214	1.414214	5.099020	7.071
1	1.0	3.0	0.0	0.000000	3.0	0.000000	4.123106	4.123106	2.236068	4.123
2	2.0	6.0	0.0	0.000000	3.0	0.000000	7.071068	7.071068	1.414214	1.414
3	3.0	9.0	0.0	0.000000	3.0	1.570796	NaN	NaN	4.123106	2.236
4	4.0	12.0	0.0	1.570796	2.5	0.000000	NaN	NaN	7.071068	5.099
5	5.0	12.0	2.5	1.570796	2.5	1.570796	NaN	NaN	7.158911	5.220
6	6.0	12.0	5.0	-3.141593	3.0	0.000000	NaN	NaN	8.062258	6.403
7	7.0	9.0	5.0	-3.141593	3.0	0.000000	NaN	NaN	5.656854	4.472
8	8.0	6.0	5.0	-3.141593	3.0	0.000000	9.219544	8.062258	4.123106	4.123
9	9.0	3.0	5.0	-3.141593	3.0	0.000000	7.211103	5.656854	4.472136	5.656
10	10.0	0.0	5.0	-3.141593	0.0	0.000000	6.082763	4.123106	6.403124	8.062

11 rows × 30 columns

Adding noise

The data provided is ground truth. The code below adds the required noise. You can change these, but you should put everything back to the original settings when you submit

In [53]:

```
## Code to roll out the trajectory

def step_robot_pose(pose_curr: np.ndarray, speed: float, delta_theta: float)
    """
    Take the speed and turn to update the 2D pose.
    """
    # Unpack the pose
```

```
    x, y, theta = pose_curr

    # Update the position
    x += speed * np.cos(theta)
    y += speed * np.sin(theta)

    # Update the angle at the END of the step
    theta += delta_theta

    # Wrap the angle to [-pi, pi]
    theta = (theta + np.pi) % (2 * np.pi) - np.pi

    # Pack the pose back up
    return np.array([x, y, theta])

def rollout_robot_pose(pose_start: np.ndarray,
                       speeds: np.ndarray,
                       delta_thetas: np.ndarray,
                       ignore_last: bool = True):
    """
    Take the starting pose and step multiple times to get a roll out from th
    starting pose.
    """
    pose_list = [pose_start]
    pose_curr = pose_start.copy()

    for speed, delta_theta in zip(speeds, delta_thetas):
        pose_curr = step_robot_pose(pose_curr, speed, delta_theta)
        pose_list.append(pose_curr)

    if ignore_last:
        pose_list.pop()

    return np.array(pose_list)
```

In [54]:

```
# Create noise models

# For the submission use noise_factor = 1, but you may find it helpful to us
# noise_factor = 1e-3 # for testing your code.
noise_factor = 1

# NUMPY VERSIONS
# Feel free to change the numbers, just be sure to change them back for the
# homework submission.
# We assume "pretty good starting"
PRIOR_NOISE_NUMPY = noise_factor * np.array([0.1, 0.1, np.deg2rad(1)])
# 0.3 units in x and y, and 20 degree error in angle, perhaps from wheel enc
ODOMETRY_NOISE_NUMPY = noise_factor * np.array([0.3, 0.3, np.deg2rad(20)])
# We assume the LIDAR measurements are a lot cleaner
# 0.01 in x and y, and 0.01 degree error in angle
MEASUREMENT_NOISE_NUMPY =  noise_factor * np.array([0.01, np.deg2rad(0.01)])

# GTSAM versions

PRIOR_NOISE = gtsam.noiseModel.Diagonal.Sigmas(noise_factor * PRIOR_NOISE_NU
```

```
ODOMETRY_NOISE = gtsam.noiseModel.Diagonal.Sigmas(noise_factor * ODOMETRY_NC
MEASUREMENT_NOISE = gtsam.noiseModel.Diagonal.Sigmas(noise_factor * MEASUREM
```

In [55]:
```
# Make a dataframe but with the noisy data
robot_noisy_df = robot_gt_df.copy()

# The random seed is for sanity, to make sure you can run the notebook agair
# and get the exact same plots.
np.random.seed(273)
robot_noisy_df["speed"] += np.random.normal(0, ODOMETRY_NOISE_NUMPY[0], n_ti
robot_noisy_df["dtheta"] += np.random.normal(0, ODOMETRY_NOISE_NUMPY[-1], n_

# again, we are using "near-perfect initialization" or "relative to initiali
# So, no noise is added to the initial pose
inital_pose = robot_gt_df[["x", "y", "theta"]].iloc[0].to_numpy()
robot_noisy_df[["x", "y", "theta"]] = rollout_robot_pose(
    inital_pose, robot_noisy_df["speed"], robot_noisy_df["dtheta"])

# add noise to the measurements
# Loop through the landmarks for range and bearing
for l_ind in range(n_landmarks):
    robot_gt_df[f"range_{l_ind}"] += np.random.normal(0, MEASUREMENT_NOISE_N
    robot_gt_df[f"bearing_{l_ind}"] += np.random.normal(0, MEASUREMENT_NOISE

robot_noisy_df
```

Out[55]:

	# time	x	y	theta	speed	dtheta	range_0	range_1
0	0.0	0.000000	0.000000	0.000000	3.254924	-0.070605	1.414214	1.414214
1	1.0	3.254924	0.000000	-0.070605	3.008145	-0.210278	4.123106	4.123106
2	2.0	6.255575	-0.212213	-0.280883	3.222176	0.330512	7.071068	7.071068
3	3.0	9.351478	-1.105413	0.049630	3.310150	2.089231	NaN	NaN
4	4.0	12.657552	-0.941200	2.138860	2.482692	-0.335601	NaN	NaN
5	5.0	11.321861	1.151571	1.803259	2.678351	1.677509	NaN	NaN
6	6.0	10.704836	3.757880	-2.802417	3.001207	-0.221030	NaN	NaN
7	7.0	7.874610	2.759347	-3.023447	2.853366	0.082388	NaN	NaN
8	8.0	5.041135	2.423018	-2.941059	2.969787	-0.219678	9.219544	8.062258
9	9.0	2.130861	1.831459	3.122448	2.694623	0.190524	7.211103	5.656854
10	10.0	-0.563267	1.883044	-2.970214	-0.454075	0.975938	6.082763	4.123106

11 rows × 30 columns

(Q4.b-c) Building the factor graph

In [56]:
```
# Re-import here so that you can rerun this cell to debug without rerunning
# everything
from gtsam.symbol_shorthand import L, X

# Create an empty nonlinear factor graph
### TODO (Q4.b) ###
graph = gtsam.NonlinearFactorGraph()
###

Xs = [X(i) for i in range(n_time_steps)]
Ls = [L(i) for i in range(n_landmarks)]

# Add a prior factor at the origin to set the origin of the SLAM problem
### TODO (Q4.b) ###
pose0 = gtsam.Pose2(0, 0, 0)
graph.add(gtsam.PriorFactorPose2(Xs[0], pose0, PRIOR_NOISE))

###

# Loop through the noisy data (i.e., `robot_noisy_df`)
# In the loop,
#    get the relative motion with pose_{i} - pose_{i - 1}
#    the measurements are in the noisy data
# Don't forget the loop closure constraint
### TODO (Q4.c) ####
prev_pose = pose0

rel_motion = []

for i,row in robot_noisy_df.iterrows():
    # print(row.keys())
    if i == 0:
        continue

    # Relative motion pose data
    cur_pose = gtsam.Pose2(*row[["x", "y", "theta"]])
    graph.add(gtsam.BetweenFactorPose2(Xs[i-1], Xs[i], prev_pose.between(cur
    prev_pose = cur_pose

    for i_l in range(n_landmarks):
        l_range = robot_noisy_df[f"range_{i_l}"][i]
        l_bearing = robot_noisy_df[f"bearing_{i_l}"][i]

        if pd.isnull(l_range) or pd.isnull(l_bearing):
            continue
        # print(f"pose {i} landmark {i_l} - bearing: {l_bearing}\n\t     l_r
        graph.add(gtsam.BearingRangeFactor2D(Xs[i], Ls[i_l],
            gtsam.Rot2.fromDegrees(l_bearing), l_range, MEASUREMENT_NOISE))

# Loop closure constraint
graph.add(gtsam.BetweenFactorPose2(Xs[2], Xs[8], gtsam.Pose2(0, 5, 180), ODO

###

# Print the factor graph to see all the nodes
```

```
### TODO (Q4.c) ####
print(graph)

###
```

```
NonlinearFactorGraph: size: 106

Factor 0: PriorFactor on x0
  prior mean:  (0, 0, 0)
  noise model: diagonal sigmas [0.1; 0.1; 0.0174532925];

Factor 1: BetweenFactor(x0,x1)
  measured:  (3.25492435, 0, -0.0706048005)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 2: BearingRangeFactor
Factor 2:   keys = { x1 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -2.89661399
range  4.12310563

Factor 3: BearingRangeFactor
Factor 3:   keys = { x1 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.89661399
range  4.12310563

Factor 4: BearingRangeFactor
Factor 4:   keys = { x1 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.463647609
range  2.23606798

Factor 5: BearingRangeFactor
Factor 5:   keys = { x1 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.244978663
range  4.12310563

Factor 6: BearingRangeFactor
Factor 6:   keys = { x1 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.141897055
range  7.07106781

Factor 7: BearingRangeFactor
Factor 7:   keys = { x1 l5 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -0.110657221
range  9.05538514

Factor 8: BearingRangeFactor
Factor 8:   keys = { x1 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.519146114
range  8.06225775

Factor 9: BearingRangeFactor
Factor 9:   keys = { x1 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
```

```
range  5.65685425

Factor 10: BearingRangeFactor
Factor 10:   keys = { x1 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.10714872
range  4.47213595

Factor 11: BearingRangeFactor
Factor 11:   keys = { x1 l10 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  5.65685425

Factor 12: BearingRangeFactor
Factor 12:   keys = { x1 l11 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.15879893
range  7.21110255

Factor 13: BetweenFactor(x1,x2)
  measured:  (3.00814534, 2.77555756e-17, -0.210278091)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 14: BearingRangeFactor
Factor 14:   keys = { x2 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -2.9996956
range  7.07106781

Factor 15: BearingRangeFactor
Factor 15:   keys = { x2 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.9996956
range  7.07106781

Factor 16: BearingRangeFactor
Factor 16:   keys = { x2 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  1.41421356

Factor 17: BearingRangeFactor
Factor 17:   keys = { x2 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  1.41421356

Factor 18: BearingRangeFactor
Factor 18:   keys = { x2 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.244978663
range  4.12310563

Factor 19: BearingRangeFactor
Factor 19:   keys = { x2 l5 }
```

```
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -0.165148677
range  6.08276253

Factor 20: BearingRangeFactor
Factor 20:   keys = { x2 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  8.48528137

Factor 21: BearingRangeFactor
Factor 21:   keys = { x2 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  5.65685425

Factor 22: BearingRangeFactor
Factor 22:   keys = { x2 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.32581766
range  4.12310563

Factor 23: BearingRangeFactor
Factor 23:   keys = { x2 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.81577499
range  4.12310563

Factor 24: BearingRangeFactor
Factor 24:   keys = { x2 l10 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.62244654
range  8.06225775

Factor 25: BearingRangeFactor
Factor 25:   keys = { x2 l11 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.43296638
range  9.21954446

Factor 26: BetweenFactor(x2,x3)
  measured:  (3.22217637, 2.22044605e-16, 0.330512456)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 27: BearingRangeFactor
Factor 27:   keys = { x3 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.89661399
range  4.12310563

Factor 28: BearingRangeFactor
Factor 28:   keys = { x3 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.67794504
range  2.23606798
```

```
Factor 29: BearingRangeFactor
Factor 29:   keys = { x3 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  1.41421356

Factor 30: BearingRangeFactor
Factor 30:   keys = { x3 l5 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -0.321750554
range  3.16227766

Factor 31: BearingRangeFactor
Factor 31:   keys = { x3 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.10714872
range  6.70820393

Factor 32: BearingRangeFactor
Factor 32:   keys = { x3 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.32581766
range  4.12310563

Factor 33: BearingRangeFactor
Factor 33:   keys = { x3 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.03444394
range  4.47213595

Factor 34: BearingRangeFactor
Factor 34:   keys = { x3 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  5.65685425

Factor 35: BetweenFactor(x3,x4)
  measured:  (3.31014995, 2.22044605e-16, 2.08923065)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 36: BearingRangeFactor
Factor 36:   keys = { x4 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.42889927
range  7.07106781

Factor 37: BearingRangeFactor
Factor 37:   keys = { x4 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.37340077
range  5.09901951

Factor 38: BearingRangeFactor
Factor 38:   keys = { x4 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.10714872
```

```
range  2.23606798

Factor 39: BearingRangeFactor
Factor 39:   keys = { x4 l5 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -3.14159265
range  1

Factor 40: BearingRangeFactor
Factor 40:   keys = { x4 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0
range  6

Factor 41: BearingRangeFactor
Factor 41:   keys = { x4 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.463647609
range  4.47213595

Factor 42: BearingRangeFactor
Factor 42:   keys = { x4 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.896055385
range  6.40312424

Factor 43: BearingRangeFactor
Factor 43:   keys = { x4 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.05165021
range  8.06225775

Factor 44: BetweenFactor(x4,x5)
  measured:  (2.48269205, 0, -0.335600897)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 45: BearingRangeFactor
Factor 45:   keys = { x5 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.78188966
range  7.15891053

Factor 46: BearingRangeFactor
Factor 46:   keys = { x5 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.86225312
range  5.22015325

Factor 47: BearingRangeFactor
Factor 47:   keys = { x5 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.21429744
range  2.5

Factor 48: BearingRangeFactor
Factor 48:   keys = { x5 l5 }
```

```
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -3.14159265
range  3.5

Factor 49: BearingRangeFactor
Factor 49:   keys = { x5 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0
range  3.5

Factor 50: BearingRangeFactor
Factor 50:   keys = { x5 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.927295218
range  2.5

Factor 51: BearingRangeFactor
Factor 51:   keys = { x5 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.27933953
range  5.22015325

Factor 52: BearingRangeFactor
Factor 52:   keys = { x5 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.35970299
range  7.15891053

Factor 53: BetweenFactor(x5,x6)
  measured:  (2.67835095, 0, 1.67750945)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 54: BearingRangeFactor
Factor 54:   keys = { x6 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.519146114
range  8.06225775

Factor 55: BearingRangeFactor
Factor 55:   keys = { x6 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.674740942
range  6.40312424

Factor 56: BearingRangeFactor
Factor 56:   keys = { x6 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.10714872
range  4.47213595

Factor 57: BearingRangeFactor
Factor 57:   keys = { x6 l5 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.57079633
range  6
```

```
Factor 58: BearingRangeFactor
Factor 58:   keys = { x6 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -1.57079633
range  1

Factor 59: BearingRangeFactor
Factor 59:   keys = { x6 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.463647609
range  2.23606798

Factor 60: BearingRangeFactor
Factor 60:   keys = { x6 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.19739556
range  5.09901951

Factor 61: BearingRangeFactor
Factor 61:   keys = { x6 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.141897055
range  7.07106781

Factor 62: BetweenFactor(x6,x7)
  measured:  (3.0012071, -3.78169718e-16, -0.221030251)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 63: BearingRangeFactor
Factor 63:   keys = { x7 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  5.65685425

Factor 64: BearingRangeFactor
Factor 64:   keys = { x7 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.10714872
range  4.47213595

Factor 65: BearingRangeFactor
Factor 65:   keys = { x7 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.81577499
range  4.12310563

Factor 66: BearingRangeFactor
Factor 66:   keys = { x7 l5 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.03444394
range  6.70820393

Factor 67: BearingRangeFactor
Factor 67:   keys = { x7 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -2.8198421
```

```
range  3.16227766

Factor 68: BearingRangeFactor
Factor 68:   keys = { x7 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  1.41421356

Factor 69: BearingRangeFactor
Factor 69:   keys = { x7 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.463647609
range  2.23606798

Factor 70: BearingRangeFactor
Factor 70:   keys = { x7 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.244978663
range  4.12310563

Factor 71: BetweenFactor(x7,x8)
  measured:  (2.85336622, -4.4408921e-16, 0.08238768)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 72: BearingRangeFactor
Factor 72:   keys = { x8 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.708626272
range  9.21954446

Factor 73: BearingRangeFactor
Factor 73:   keys = { x8 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.519146114
range  8.06225775

Factor 74: BearingRangeFactor
Factor 74:   keys = { x8 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.32581766
range  4.12310563

Factor 75: BearingRangeFactor
Factor 75:   keys = { x8 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.81577499
range  4.12310563

Factor 76: BearingRangeFactor
Factor 76:   keys = { x8 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  5.65685425

Factor 77: BearingRangeFactor
Factor 77:   keys = { x8 l5 }
```

```
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  8.48528137

Factor 78: BearingRangeFactor
Factor 78:   keys = { x8 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -2.97644398
range  6.08276253

Factor 79: BearingRangeFactor
Factor 79:   keys = { x8 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.89661399
range  4.12310563

Factor 80: BearingRangeFactor
Factor 80:   keys = { x8 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  1.41421356

Factor 81: BearingRangeFactor
Factor 81:   keys = { x8 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  1.41421356

Factor 82: BearingRangeFactor
Factor 82:   keys = { x8 l10 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.141897055
range  7.07106781

Factor 83: BearingRangeFactor
Factor 83:   keys = { x8 l11 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -0.141897055
range  7.07106781

Factor 84: BetweenFactor(x8,x9)
  measured:  (2.96978709, 0, -0.219678251)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 85: BearingRangeFactor
Factor 85:   keys = { x9 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.982793723
range  7.21110255

Factor 86: BearingRangeFactor
Factor 86:   keys = { x9 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  5.65685425
```

```
Factor 87: BearingRangeFactor
Factor 87:   keys = { x9 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.03444394
range  4.47213595

Factor 88: BearingRangeFactor
Factor 88:   keys = { x9 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.35619449
range  5.65685425

Factor 89: BearingRangeFactor
Factor 89:   keys = { x9 l4 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.62244654
range  8.06225775

Factor 90: BearingRangeFactor
Factor 90:   keys = { x9 l6 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -3.03093543
range  9.05538514

Factor 91: BearingRangeFactor
Factor 91:   keys = { x9 l7 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.9996956
range  7.07106781

Factor 92: BearingRangeFactor
Factor 92:   keys = { x9 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.89661399
range  4.12310563

Factor 93: BearingRangeFactor
Factor 93:   keys = { x9 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.67794504
range  2.23606798

Factor 94: BearingRangeFactor
Factor 94:   keys = { x9 l10 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.244978663
range  4.12310563

Factor 95: BearingRangeFactor
Factor 95:   keys = { x9 l11 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -0.244978663
range  4.12310563

Factor 96: BetweenFactor(x9,x10)
  measured:  (2.69462262, 0, 0.190523537)
```

```
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];

Factor 97: BearingRangeFactor
Factor 97:   keys = { x10 l0 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.40564765
range  6.08276253

Factor 98: BearingRangeFactor
Factor 98:   keys = { x10 l1 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 1.32581766
range  4.12310563

Factor 99: BearingRangeFactor
Factor 99:   keys = { x10 l2 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.46685171
range  6.40312424

Factor 100: BearingRangeFactor
Factor 100:   keys = { x10 l3 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.62244654
range  8.06225775

Factor 101: BearingRangeFactor
Factor 101:   keys = { x10 l8 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.9996956
range  7.07106781

Factor 102: BearingRangeFactor
Factor 102:   keys = { x10 l9 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 2.94419709
range  5.09901951

Factor 103: BearingRangeFactor
Factor 103:   keys = { x10 l10 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : 0.785398163
range  1.41421356

Factor 104: BearingRangeFactor
Factor 104:   keys = { x10 l11 }
  noise model: diagonal sigmas [0.01; 0.000174532925];
ExpressionFactor with measurement: bearing : -0.785398163
range  1.41421356

Factor 105: BetweenFactor(x2,x8)
  measured:  (0, 5, -2.21237391)
  noise model: diagonal sigmas [0.3; 0.3; 0.34906585];
```

(Q4.d) Initialize the optimization problem

In [57]:
```python
# Set-up a values data structure for the initial estimate
### TODO (Q4.d) ###
initial_estimate = gtsam.Values()

###

# Set the initial poses from the noisy odometry alone
# Hint, you already have this in `robot_noisy_df`
### TODO (Q4.d) ###

for i,row in robot_noisy_df.iterrows():
    pose = gtsam.Pose2(*row[["x", "y", "theta"]])
    initial_estimate.insert(Xs[i], pose)

###

# Sample random values for the initial landmark positions
# In reality, you would have to estimate these from odometry,
# but to not over-complicate the problem, just use noisy
# ground-truth.
# In reality, the initialization is very important for the
# graph optimization to converge to a good solution!!
l_init_vec = [
    (-1, -1),
    (-1, 1),
    (5, 1),
    (7, 1),
    (10, 1),
    (12, -1),
    (12, 6),
    (10, 4),
    (7, 4),
    (5, 4),
    (-1, 4),
    (-1, 6)
]

observed_landmarks = set()

for l_ind, L_val in enumerate(Ls):
    l_hat = l_init_vec[l_ind]
    point_init = (np.random.normal(l_hat[0], ODOMETRY_NOISE_NUMPY[0]),
                  np.random.normal(l_hat[1], ODOMETRY_NOISE_NUMPY[0]))
    # Add the initial estimates of the landmarks
    ### TODO (Q4.d) ###

    l_range = robot_noisy_df[f"range_{l_ind}"]
    l_bearing = robot_noisy_df[f"bearing_{l_ind}"]

    # print(l_bearing.notna().any())
    was_observed = (
        l_range.notna().any() and
```

```
        l_bearing.notna().any()
    )

    # if was_observed:
    #     initial_estimate.insert(...)

    if was_observed:
        print(l_ind)
        landmark = gtsam.Point2(*point_init)
        initial_estimate.insert(L_val, landmark)
        observed_landmarks.add(l_ind)


# Print the initial estimates to verify
### TODO (Q4.d) ###
initial_estimate
###
```

```
0
1
2
3
4
5
6
7
8
9
10
11
```

Out[57]:
```
Values with 23 values:
Value l0: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -1.04531412;
        -1.21481369
]

Value l1: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -1.00629039;
        1.17309668
]

Value l2: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        4.5809314;
        0.992908941
]

Value l3: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        6.58246277;
        0.865267077
]

Value l4: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        9.66434412;
        1.08988328
]

Value l5: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        11.5884324;
        -1.3857714
]

Value l6: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        12.4511261;
        5.92072691
]

Value l7: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        9.46084069;
        3.89537827
]

Value l8: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        7.25516174;
        4.41311515
]

Value l9: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
```

```
[
        4.81397342;
        3.97336983
]

Value l10: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -1.36212978;
        3.67352319
]

Value l11: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -0.498201812;
        5.55854322
]

Value x0: (gtsam::Pose2)
(0, 0, 0)

Value x1: (gtsam::Pose2)
(3.25492435, 0, -0.0706048005)

Value x2: (gtsam::Pose2)
(6.25557495, -0.212213084, -0.280882892)

Value x3: (gtsam::Pose2)
(9.35147768, -1.10541342, 0.0496295644)

Value x4: (gtsam::Pose2)
(12.6575519, -0.94119955, 2.13886021)

Value x5: (gtsam::Pose2)
(11.321861, 1.15157128, 1.80325932)

Value x6: (gtsam::Pose2)
(10.704836, 3.75787977, -2.80241654)

Value x7: (gtsam::Pose2)
(7.87461038, 2.75934738, -3.02344679)

Value x8: (gtsam::Pose2)
(5.04113528, 2.42301767, -2.94105911)

Value x9: (gtsam::Pose2)
(2.13086141, 1.83145921, 3.12244795)

Value x10: (gtsam::Pose2)
(-0.563267406, 1.88304381, -2.97021382)
```

In [58]:
```
# Plot the initial poses and landmarks.
# For example, for poses, the code is
# gtsam_plot.plot_pose2(
#         fignum: int,
#         pose: Pose2,
#         axis_length: float = 0.1,
```

```
#         covariance: np.ndarray = None,
#         axis_labels=("X axis", "Y axis", "Z axis"),
# )
# We do need covariance for this part. The source code is here:
# https://github.com/borglab/gtsam/blob/develop/python/gtsam/utils/plot.py

# Don't forget to use axis equals!


### TODO (Q4.d) ###

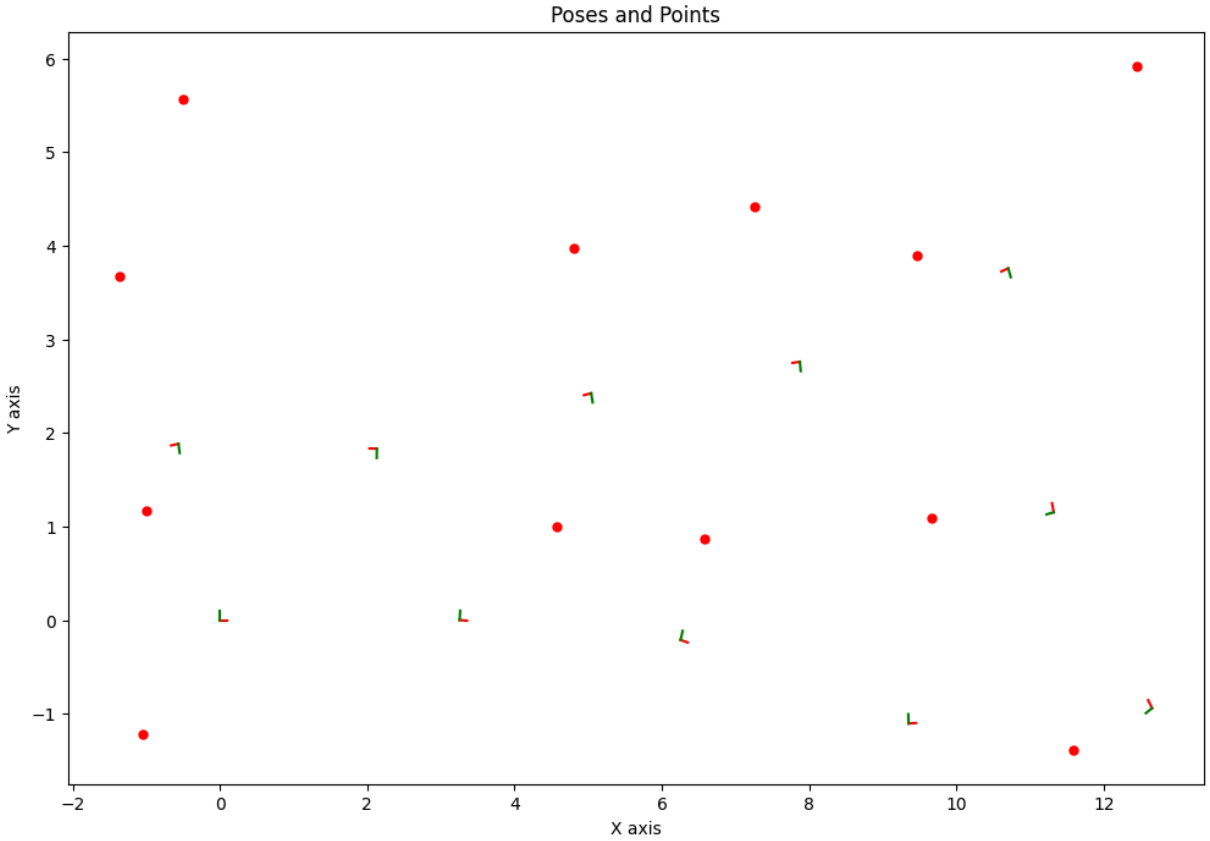
fignum = 0
plt.figure(fignum, figsize=(12,8))
# Poses
for i in range(len(Xs)):
    pose = initial_estimate.atPose2(Xs[i])
    gtsam_plot.plot_pose2(fignum, pose)

# Landmarks
# for i in range(len(Ls)):
for i in observed_landmarks:
    # print(observed_landmarks)
    point = initial_estimate.atPoint2(Ls[i])
    gtsam_plot.plot_point2(fignum, point, "r.")

plt.title("Poses and Points")
plt.show()

###
```

```
/Library/Frameworks/Python.framework/Versions/3.12/lib/python3.12/site-packa
ges/gtsam/utils/plot.py:179: UserWarning: marker is redundantly defined by t
he 'marker' keyword argument and the fmt string "r." (-> marker='.'). The ke
yword argument will take precedence.
  axes.plot([point[0]], [point[1]], linespec, marker='.', markersize=10)
```



(Q4.e) Graph Optimization

In [59]:
```
# We are going to optimize with the Levenberg-Marquardt solver.
# You can definitely consider other options (e.g., Gauss Newton).
# You may be interested in AA 222: Optimization for more about these
# methods. Levenberg-Marquardt has a number of configuration
# parameters, but we will use the defaults. For your projects, you
# might consider tweaking the parameters for tighter convergence.

lm_params = gtsam.LevenbergMarquardtParams()

# for j in range(n_landmarks):
#     key = L(j)
#     if initial_estimate.exists(key) and not graph.exists(key):
#         print(j)
#         initial_estimate.erase(key)

if running_dataset_3:
    initial_estimate.erase(Ls[0])
    initial_estimate.erase(Ls[1])
# initial_estimate.erase(key)

# Fill the optimizer with your graph and initial estimates (in that
# order).
### TODO (Q4.e) ###
optimizer = gtsam.LevenbergMarquardtOptimizer(graph, initial_estimate, lm_pa
result = optimizer.optimize()
###
```

```
# Print the results
### TODO (Q4.e) ###
result
###
```

Out[59]:
```
Values with 23 values:
Value l0: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -0.805666715;
        -0.715262369
]

Value l1: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -0.664544176;
        1.27975253
]

Value l2: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        5.32050053;
        0.856384907
]

Value l3: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        7.31551542;
        0.715262361
]

Value l4: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        10.3080378;
        0.503578575
]

Value l5: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        12.1619301;
        -1.63255885
]

Value l6: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        12.655859;
        5.34999328
]

Value l7: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        10.5197216;
        3.49610091
]

Value l8: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        7.52719922;
        3.70778471
]

Value l9: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
```

```
[
        5.53218433;
        3.84890724
]

Value l10: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -0.452860364;
        4.27227488
]

Value l11: (Eigen::Matrix<double, -1, 1, 0, -1, 1>)
[
        -0.311737838;
        6.26728974
]

Value x0: (gtsam::Pose2)
(7.57412256e-16, -2.32258888e-17, -1.90091872e-16)

Value x1: (gtsam::Pose2)
(3.25492435, -8.36683809e-16, -0.0706048005)

Value x2: (gtsam::Pose2)
(6.24744668, -0.211683801, -0.0707002665)

Value x3: (gtsam::Pose2)
(9.23996905, -0.423367607, -0.0705752585)

Value x4: (gtsam::Pose2)
(12.2324914, -0.635051397, 1.50026588)

Value x5: (gtsam::Pose2)
(12.4088946, 1.85871719, 1.50013764)

Value x6: (gtsam::Pose2)
(12.5852977, 4.35248586, 3.07100617)

Value x7: (gtsam::Pose2)
(9.59277537, 4.56416963, 3.07093662)

Value x8: (gtsam::Pose2)
(6.60025303, 4.77585342, 3.07105864)

Value x9: (gtsam::Pose2)
(3.60773069, 4.98753723, 3.07092463)

Value x10: (gtsam::Pose2)
(0.615208387, 5.19922104, 3.07099224)
```

In [60]:
```
# To understand uncertainty, you need the marginal Gaussian distributions.
# gtsam makes this directly accessible with
# marginals = gtsam.Marginals(graph, result)
# The covariance matrices are in marginals. For example, the covariance
# matrix for X(1) (i.e., Xs[0]) is
# marginals.marginalCovariance(X(1))
```

```
# Check that you can print the marginal covariance matrices for both poses
# and landmarks
### TODO (Q4.e) ###

marginals = gtsam.Marginals(graph, result)
# print(marginals.marginalCovariance(X(1)))
# print(marginals.marginalCovariance(L(1)))

###
```

In [61]:

```
# Now plot the poses and landmarks _with_ covariance ellipses.
### TODO (Q4.e) ###

fignum = 1
plt.figure(fignum, figsize=(12,8))
# Poses
for i in range(len(Xs)):
    pose = initial_estimate.atPose2(Xs[i])
    cov = marginals.marginalCovariance(Xs[i])  # 3x3
    gtsam_plot.plot_pose2(fignum, pose, covariance=cov)
    plt.text(pose.x()+0.1, pose.y()+0.1, f"x{i}", fontsize=8)

# Landmarks
for i in range(len(Ls)):
    if running_dataset_3:
        if i in [0,1]:
            continue
    point = initial_estimate.atPoint2(Ls[i])
    gtsam_plot.plot_point2(fignum, point, "r.")

###

# You may find if helpful to plot line segments connecting poses at
# adjacent time steps to understand what is happening.
# For a given `pose`, use `pose.translation()` to get the translation
# term (i.e., the position)
### TODO (Q4.h) ###

plt.title("Poses and Points with Marginal Covariances")


for i in range(len(Xs) - 1):

    pose_i = initial_estimate.atPose2(Xs[i])
    pose_j = initial_estimate.atPose2(Xs[i+1])

    t_i = pose_i.translation()
    t_j = pose_j.translation()

    x_vals = [t_i[0], t_j[0]]
    y_vals = [t_i[1], t_j[1]]

    # label = "Trajectory" if i == 0 else None
    plt.plot(x_vals, y_vals, "tab:blue", label=None)
```

```
plt.legend()
plt.show()
###
```

```
/var/folders/l4/vmh1rmg52flbd1tpkc0s470c0000gn/T/ipykernel_88975/3857811003.
py:47: UserWarning: No artists with labels found to put in legend.  Note tha
t artists whose label start with an underscore are ignored when legend() is
called with no argument.
  plt.legend()
```

